

Stacks



Abstract Data Types

public
↓

private
↓

- Abstraction: separating the interface of our programs from the implementation.

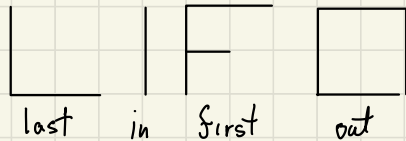
1. Procedural: steps treated as units.
2. Data: a way to store values w/o worrying how it's stored.

Stack

```
def fun_1():  
    print("Function 1")  
  
def fun_2():  
    print("Function 2")  
    fun_1()  
  
def fun_3():  
    print("Function 3")  
    fun_2()  
  
fun_3()
```



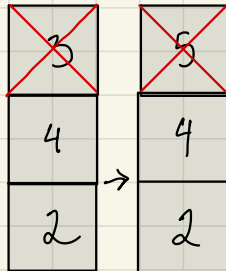
Call stack



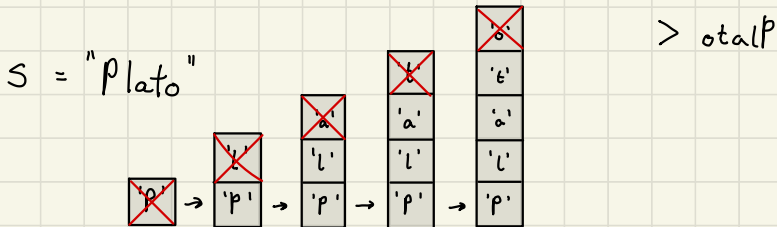
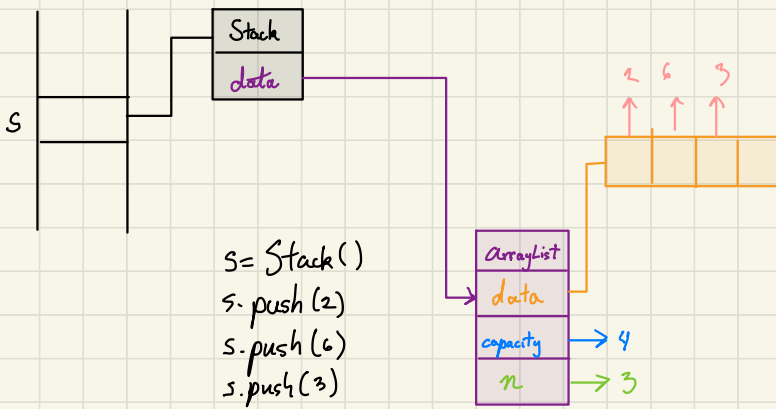
- $\Theta(1)$ 1. `stack = Stack()` # creates empty stack
- $\Theta(1)$ 2. `len(stack)` # num of items in stack
- $\Theta(1)$ 3. `stack.is_empty()` # True if `len(s) == 0` else False
- $\Theta_A(1)$ 4. `stack.push(item)` # Adds item at top/end of stack
- $\Theta_A(1)$ 5. `stack.pop()` # removes & return top/last element
- $\Theta(1)$ 6. `stack.top()` # returns top/last element

```
s = Stack()  
s.push(2)  
s.push(4)  
s.push(3)  
print(s.pop())
```

```
s.push(5)  
print(s.pop())  
print(len(s))
```



> 3
> 5
> 2



```

def reverse(string):
    stack = ArrayStack() # O(1)

    for char in string:
        stack.push(char) # O(1)

    while not stack.is_empty():
        char = stack.pop() # O(1)
        print(char, end=") # O(1)

    print() # O(1)
  
```

Polish Notation

$$2 + 2$$

infix notation

operand_A operator operand_B

$$2 2 +$$

postfix notation

operand_A operand_B operator

$$+ 2 2$$

prefix notation

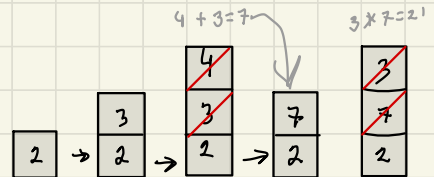
operator operand_A operand_B

$$2 \ 7 \ 3 \ * \ -$$

$$2 \ 2 \ 1 \ -$$

$$-19$$

$$2 \ 3 \ 4 \ + \ 3 \ * \ -$$



$$2 - 21 = -19$$

$$\begin{array}{|c|} \hline 21 \\ \hline 2 \\ \hline \end{array}$$

$$-19$$

```
def eval_postfix_expr(string):  
    lst = string.split() #  $O(n)$   
    stack = ArrayStack() #  $O(1)$ 
```

```
    for token in lst:
```

```
        if token not in "+-* /":
```

```
            stack.push(int(token))
```

```
        else:
```

```
            op_one = stack.pop()
```

```
            op_two = stack.pop()
```

```
            if token == '+':
```

```
                res = op_one + op_two
```

```
            elif token == '-':
```

```
                res = op_two - op_one
```

```
            elif token == '*':
```

```
                res = op_one * op_two
```

```
            else:
```

```
                if op_one == 0:
```

```
                    raise ZeroDivisionError
```

```
                else:
```

```
                    res = op_two / op_one
```

```
            stack.push(res)
```

```
    return stack.pop()
```